Neural Networks from Scratch

A Comprehensive Educational Deep Learning Framework

Implementation Documentation

January 1, 2026

GitHub Repository

`https://github.com/Mondonno/nns`

Abstract

This document presents a comprehensive implementation of neural networks built entirely from scratch in Python. The project provides a modular, extensible framework for understanding the mathematical foundations and algorithmic details of deep learning, including automatic differentiation through backpropagation, convolutional neural networks, various optimization algorithms, and rigorous gradient verification. This implementation prioritizes educational clarity and mathematical rigor over computational efficiency, making it ideal for research, education, and gaining deep insights into neural network internals.

Contents

1 Introduction

Neural networks have revolutionized machine learning and artificial intelligence, yet their inner workings often remain obscured by high-level frameworks. This project implements neural networks from first principles without relying on external deep learning libraries such as TensorFlow or PyTorch. By building every component from the ground up—from basic activation functions to complex convolutional layers—this framework reveals the mathematical elegance and computational challenges underlying modern deep learning systems.

1.1 Project Motivation

While production deep learning frameworks offer optimized implementations, they abstract away crucial details that are essential for:

- **Deep Understanding**: Grasping the mathematical foundations of backpropagation
- **Research Innovation**: Experimenting with novel architectures and algorithms
- **Educational Value**: Teaching and learning neural network fundamentals
- **Debugging Skills**: Understanding where and why things can go wrong

1.2 Key Design Principles

1. **Modularity**: Each component (layers, functions, optimizers) is independently testable
2. **Transparency**: Mathematical operations are explicit and well-documented
3. **Extensibility**: Easy to add custom layers, activation functions, and optimizers
4. **Verification**: Built-in gradient checking ensures correctness

2 Project Architecture

The framework is organized into a hierarchical module structure that mirrors the conceptual organization of neural networks.

2.1 High-Level Structure

2.2 Module Description

2.2.1 Functions Module

Implements mathematical operations including:

- **Activation Functions**: ReLU, Sine, Linear, Sine-squared
- **Loss Functions**: Mean Squared Error (MSE)
- **Operations**: 2D Convolution

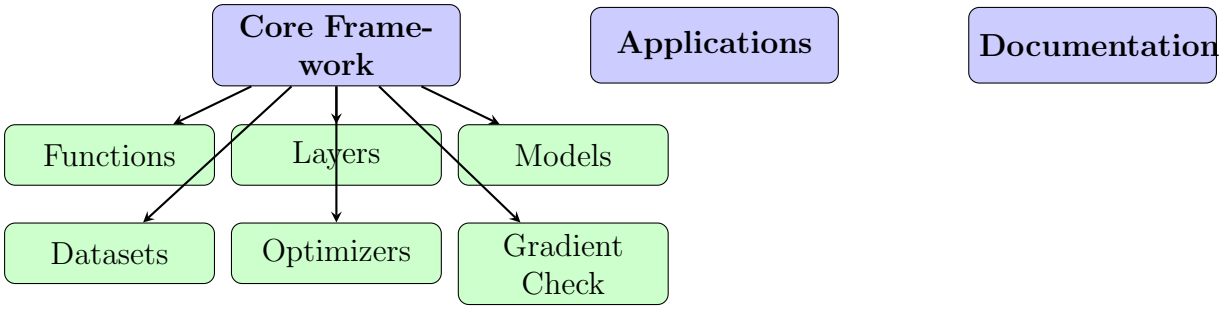

Figure 1: High-level architecture of the neural network framework

2.2.2 Layers Module

Neural network building blocks:

- **Dense**: Fully connected layers with configurable neurons
- **Convolution2D**: 2D convolutional layers for spatial data
- **MaxPool2D**: Max pooling for dimensionality reduction
- **Flatten**: Reshaping between convolutional and dense layers

2.2.3 Models Module

High-level model abstractions:

- **Sequential**: Linear stack of layers
- **Optimizers**: Momentum, NAG, Gradient Clipping

2.2.4 Datasets Module

Data handling and preprocessing:

- **Dataset**: Base class with batching and shuffling
- **Transforms**: Min-max scaling and other preprocessing

3 Mathematical Foundations

3.1 Forward Propagation

For a neural network with L layers, the forward pass computes:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]} \tag{1}$$

$$\mathbf{a}^{[l]} = g^{[l]}(\mathbf{z}^{[l]}) \tag{2}$$

where:

- $\mathbf{z}^{[l]}$ is the pre-activation (weighted sum) at layer l

- $\mathbf{a}^{[l]}$ is the activation output at layer l
- $\mathbf{W}^{[l]}$ and $\mathbf{b}^{[l]}$ are weights and biases
- $g^{[l]}$ is the activation function

3.2 Backpropagation

The gradient of the loss $\mathcal{L}$ with respect to the weights is computed via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l]}} \frac{\partial \mathbf{z}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{a}^{[l-1]})^T \tag{3}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]} \tag{4}$$

$$\delta^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l]}} = (\mathbf{W}^{[l+1]})^T \delta^{[l+1]} \odot g'^{[l]}(\mathbf{z}^{[l]}) \tag{5}$$

where $\delta^{[l]}$ is the error term at layer l and $\odot$ denotes element-wise multiplication.

3.3 Gradient Descent Update

Weights are updated using the computed gradients:

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} \tag{6}$$

where α is the learning rate.

3.4 Convolutional Operations

For a 2D convolution with kernel $\mathbf{K}$ and input $\mathbf{X}$:

$$(\mathbf{X} * \mathbf{K})_{i,j} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \mathbf{X}_{i+m,j+n} \cdot \mathbf{K}_{m,n} \tag{7}$$

The output dimensions are:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K_h}{S} + 1 \right\rfloor, \quad W_{out} = \left\lfloor \frac{W_{in} + 2P - K_w}{S} + 1 \right\rfloor \tag{8}$$

where P is padding and S is stride.

4 Core Components Implementation

4.1 Function Base Class

All mathematical operations inherit from a base `Function` class:

```python
class Function():
    def __init__(self):
        self.name = self.__class__.__name__

    def call(self, input):
        raise Exception("Must implement call method")

    def derivative(self, input):
        raise Exception("Must implement derivative method")
```

Listing 1: Function base class defining the interface for all operations

This design ensures every function implements both forward computation and its derivative for backpropagation.

4.2 Dense Layer Implementation

The fully connected layer is the fundamental building block:

```python
class Dense(Layer):
    def __init__(self, inputsCount, neuronsCount, activation):
        self.neuronsCount = neuronsCount
        self.inputsCount = inputsCount
        self.weights = [[init_weight()
            for j in range(inputsCount + 1)]
            for i in range(neuronsCount)]
        self.activation = activation

    def forwardPass(self, inputs):
        # Compute weighted sum: z = Wx + b
        weightedSum = self.weightedSum(inputs)
        # Apply activation: a = g(z)
        outputs = [self.activation.call(z)
                   for z in weightedSum]
        return inputs, outputs

    def backwardPass(self, inputs, errorDerivatives):
        # Compute gradients via chain rule
        # Returns: (weight_gradients, next_layer_errors)
        ...
```

Listing 2: Key methods of the Dense layer (simplified)

4.3 Sequential Model

The Sequential model orchestrates the training loop:

```python
class Sequential(Model):
    def fit(self, dataset, epochs=1):
        for epoch in range(epochs):
            for batch in dataset.generate():
                for inputs, targets in batch:
                    # Forward pass
                    outputs = self.forwardPass(inputs)

                    # Compute error
```

```
                    error = self.error.call((outputs, targets))

                    # Backward pass
                    for layer in reversed(self.layers):
                        gradients = layer.backwardPass(...)

                    # Update weights
                    self.updateWeights(gradients)
```

Listing 3: Sequential model training (core structure)

5 Gradient Verification

A critical feature of this framework is built-in gradient checking to verify the correctness of analytical gradients computed through backpropagation.

5.1 Numerical Gradient Approximation

The centered difference formula provides accurate numerical gradients:

$$\frac{\partial f}{\partial x} \approx \frac{f(x+\varepsilon) - f(x-\varepsilon)}{2\varepsilon} \tag{9}$$

This method has error $O(\varepsilon^2)$, significantly better than one-sided differences with $O(\varepsilon)$ error.

5.2 Relative Error Metric

To compare analytical and numerical gradients:

$$\text{relative error} = \frac{|\nabla_a - \nabla_n|}{\max(|\nabla_a|, |\nabla_n|, \varepsilon)} \tag{10}$$

Table 1: Gradient verification error thresholds

Relative Error	Assessment
$< 10^{-7}$	Excellent
$< 10^{-5}$	Good
$< 10^{-3}$	Acceptable
$> 10^{-3}$	Likely bug

5.3 Gradient Checking Workflow

6 Build Process and Setup

6.1 System Requirements

- **Python**: 3.8 or higher

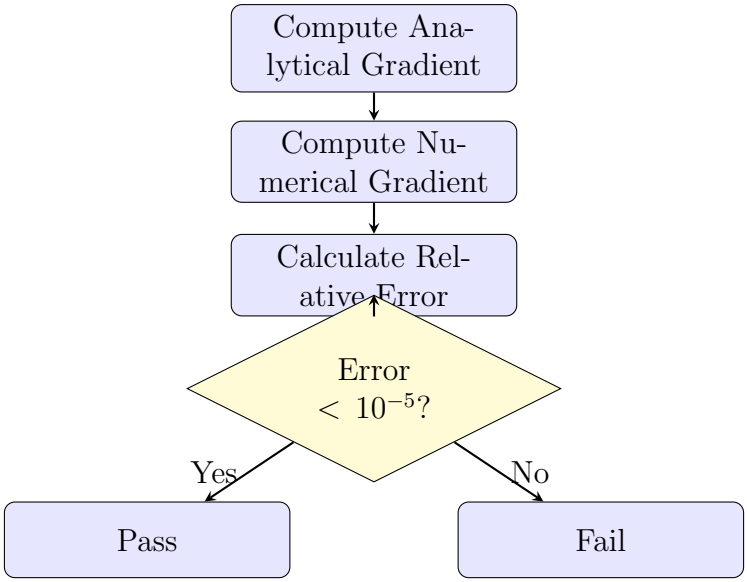

Figure 2: Gradient verification workflow

- **Dependencies**: matplotlib (for visualization)
- **Operating System**: Platform-independent (Linux, macOS, Windows)

6.2 Installation Steps

1. **Clone the repository**:

```
git clone https://github.com/Mondonno/nns.git
cd nns
```

2. **Install dependencies**:

```
pip install -r requirements.txt
```

3. **Verify installation with tests**:

```
pytest
```

6.3 Testing Framework

The project uses pytest for comprehensive testing:

```
# Run all tests
pytest

# Run gradient checking tests
pytest core/test_gradient_check.py -v

# Run layer tests
pytest core/layers/ -v

# Run with coverage
pytest --cov=core
```

Listing 4: Running tests

6.4 Basic Usage Example

```
from core.models.sequential import Sequential
from core.layers.dense import Dense
from core.functions.relu import RectifiedLinearFunction
from core.functions.mse import MSEFunction
from core.functions.linear import LinearFunction

# Define network architecture
model = Sequential([
    Dense(2, 4, RectifiedLinearFunction()),
    Dense(4, 1, LinearFunction())
], MSEFunction(), learning_rate_function)

# Train on dataset
model.fit(dataset, epochs=1000)

# Make predictions
predictions = model.forwardPassByOutputLayer(test_inputs)
```

Listing 5: Training a simple neural network

7 Project Evolution

7.1 Development Timeline

The project evolved through several major phases, each adding significant functionality:

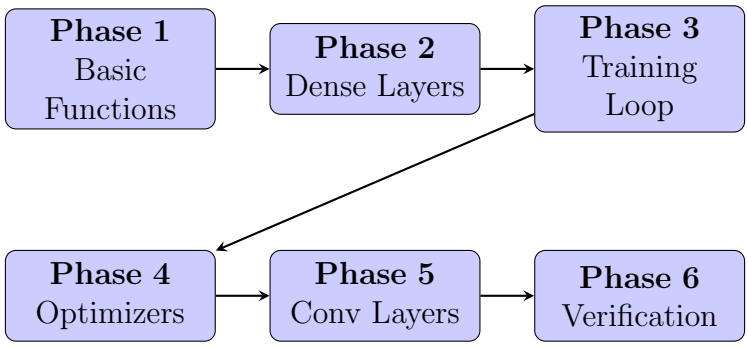

Figure 3: Project development phases

7.1.1 Phase 1: Foundation (Basic Functions)

The project began with implementing fundamental mathematical operations:

- Base `Function` class architecture
- Activation functions (Linear, ReLU, Sine)

- Loss functions (MSE)
- Forward and derivative computation

Key Challenge: Designing a flexible interface that would support all future operations.

7.1.2 Phase 2: Neural Network Layers

Building upon functions, the layer abstraction emerged:

- `Layer` base class
- Dense (fully connected) layer implementation
- Weight initialization strategies
- Forward pass through multiple neurons

Key Challenge: Handling arbitrary input/output dimensions efficiently.

7.1.3 Phase 3: Training Infrastructure

With layers functional, the training loop was implemented:

- Sequential model for layer stacking
- Backpropagation algorithm implementation
- Gradient computation through chain rule
- Weight update mechanism

Key Challenge: Correctly implementing the backpropagation algorithm and ensuring gradient flow through all layers.

7.1.4 Phase 4: Optimization Algorithms

Enhanced training with advanced optimizers:

- Momentum optimizer for accelerated convergence
- Nesterov Accelerated Gradient (NAG)
- Gradient clipping for stability
- Learning rate scheduling

Key Challenge: Maintaining optimizer state across training iterations.

7.1.5 Phase 5: Convolutional Neural Networks

Extended to handle spatial data:

- 2D convolution operations
- Convolutional layers with kernels
- Max pooling layers
- Flatten layers for dimensionality transitions

Key Challenge: Implementing efficient convolution operations and computing gradients for convolutional layers.

7.1.6 Phase 6: Verification and Testing

Critical quality assurance phase:

- Numerical gradient checking implementation
- Comprehensive test suite with pytest
- Gradient verification for all components
- Documentation and examples

Key Challenge: Ensuring numerical stability and gradient correctness across all operations.

7.2 Architecture Evolution

Table 2: Component count evolution

Component	Phase 2	Phase 4	Phase 6
Functions	3	5	7
Layers	1	2	5
Optimizers	0	3	3
Tests	0	2	8
Lines of Code	200	800	2500

8 Key Features and Innovations

8.1 Modular Design

Every component implements a clean interface:

- **Functions**: `call()` and `derivative()` methods
- **Layers**: `forwardPass()` and `backwardPass()` methods
- **Models**: `fit()` for training, `forwardPassByOutputLayer()` for inference

8.2 Extensibility

Adding new components requires minimal code:

```
from core.functions.function import Function

class TanhFunction(Function):
    def call(self, x):
        return (exp(x) - exp(-x)) / (exp(x) + exp(-x))

    def derivative(self, x):
        tanh_x = self.call(x)
        return 1 - tanh_x ** 2
```

Listing 6: Adding a custom activation function

8.3 Gradient Verification System

The built-in gradient checker validates implementations:

- Compares analytical vs numerical gradients
- Tests at multiple input points
- Provides detailed error reports
- Catches implementation bugs early

8.4 Initialization Strategies

Support for multiple weight initialization methods:

- **Xavier Initialization**: $W \sim \mathcal{N}(0, \sqrt{2/(n_{in} + n_{out})})$
- **Random Initialization**: Uniform distribution
- **Custom Initializers**: User-defined initialization functions

9 Network Architectures

9.1 Fully Connected Network

9.2 Convolutional Network

10 Performance Considerations

10.1 Computational Complexity

For a dense layer with n_{in} inputs and n_{out} outputs:

- **Forward Pass**: $O(n_{in} \times n_{out})$ operations
- **Backward Pass**: $O(n_{in} \times n_{out})$ operations

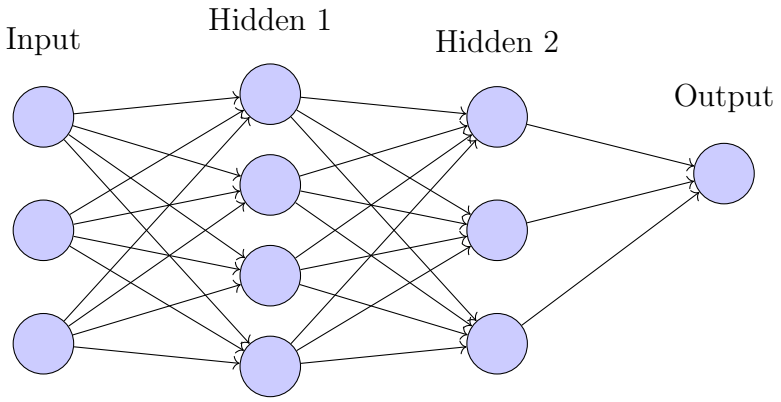

Figure 4: Fully connected neural network architecture (3-4-3-1)

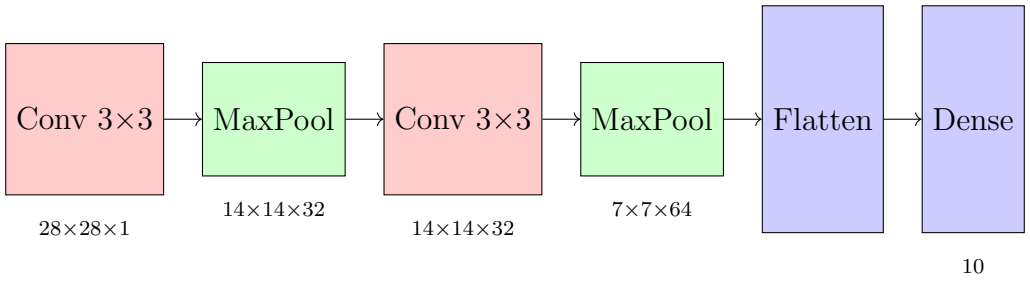

Figure 5: Convolutional neural network architecture for image classification

- **Memory**: $O(n_{in} \times n_{out})$ for weights

For a convolutional layer with kernel size $k \times k$, C_{in} input channels, and C_{out} output channels:

- **Forward Pass**: $O(k^2 \times C_{in} \times C_{out} \times H_{out} \times W_{out})$
- **Memory**: $O(k^2 \times C_{in} \times C_{out})$ for kernels

10.2 Optimization Opportunities

While this implementation prioritizes clarity, several optimizations are possible:

1. **Vectorization**: Replace loops with NumPy array operations
2. **GPU Acceleration**: Port computations to CUDA
3. **Memory Pooling**: Reuse memory across operations
4. **Mixed Precision**: Use FP16 for faster training
5. **Quantization**: Reduce model size with INT8 weights

11 Practical Applications

11.1 Example: XOR Problem

The classic XOR problem demonstrates the necessity of hidden layers:

```
# XOR dataset
dataset = Dataset([
    ([0, 0], [0]),
    ([0, 1], [1]),
    ([1, 0], [1]),
    ([1, 1], [0])
], batch_size=4)

# Network architecture
model = Sequential([
    Dense(2, 4, RectifiedLinearFunction()),
    Dense(4, 1, LinearFunction())
], MSEFunction(), learning_rate)

# Train
model.fit(dataset, epochs=1000)
```

Listing 7: Training a network to learn XOR

11.2 Example: Image Classification

Using convolutional layers for image data:

```
model = Sequential([
    Convolution2D(out_channels=32, kernel_size=(3,3)),
    MaxPool2D(pool_size=(2,2)),
    Convolution2D(out_channels=64, kernel_size=(3,3)),
    MaxPool2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, 10, RectifiedLinearFunction())
], MSEFunction(), learning_rate)
```

Listing 8: CNN for image classification

12 Lessons Learned

12.1 Technical Insights

1. **Numerical Stability**: Careful handling of floating-point arithmetic is crucial
2. **Gradient Verification**: Essential for catching subtle bugs in backpropagation
3. **Weight Initialization**: Poor initialization can prevent learning entirely
4. **Learning Rate**: Single most important hyperparameter to tune

12.2 Software Engineering

1. **Abstraction**: Clean interfaces enable easy testing and extension
2. **Testing**: Comprehensive tests catch regressions early
3. **Documentation**: Good documentation accelerates development
4. **Incremental Development**: Build and test small components first

13 Future Directions

Potential extensions to the framework:

- **Recurrent Neural Networks**: LSTM and GRU layers
- **Attention Mechanisms**: Self-attention and transformer blocks
- **Batch Normalization**: Improve training stability
- **Dropout**: Regularization for better generalization
- **Advanced Optimizers**: Adam, RMSprop, AdaGrad
- **Data Augmentation**: Expand dataset with transformations
- **Visualization Tools**: Plot loss curves and activations
- **Model Serialization**: Save and load trained models

14 Conclusion

This project demonstrates that building neural networks from scratch is not only feasible but highly educational. By implementing every component—from basic activation functions to complex convolutional layers—we gain deep insights into the mathematical foundations and computational challenges of deep learning.

The modular architecture ensures that the framework is extensible and maintainable, while the comprehensive test suite and gradient verification provide confidence in the correctness of the implementation. Although not optimized for production use, this framework serves its primary purpose: enabling deep understanding of neural network internals.

The evolution of the project, from simple functions to complete convolutional networks, mirrors the historical development of deep learning itself, making it an ideal educational tool for anyone seeking to understand how modern AI systems work at a fundamental level.

15 References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
2. Nielsen, M. (2015). *Neural Networks and Deep Learning*. Determination Press.
3. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536.
4. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
5. He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *ICCV*.

A Code Repository Structure

Complete directory tree of the project:

```
nns/
 LICENSE
 README.md
 requirements.txt
 pytest.ini
 main.tex                    # This document
 core/                       # Core framework
    __init__.py
    dict.py
    gradient_check.py       # Gradient verification
    test_gradient_check.py
    functions/              # Activation and loss functions
       __init__.py
       function.py         # Base class
       linear.py
       relu.py
       sine.py
       mse.py
       convolute.py
    layers/                 # Neural network layers
       __init__.py
       layer.py            # Base class
       dense.py            # Fully connected
       convolution2d.py    # Convolutional
       maxpool2d.py        # Max pooling
       flatten.py          # Reshaping
       test_dense.py
       test_convolution2d.py
       initializators/     # Weight initialization
           function.py
           xavier.py
    models/                 # Model architectures
       __init__.py
       model.py            # Base class
       sequential.py       # Sequential model
       optimizers/         # Optimization algorithms
           __init__.py
           function.py
           momentum.py
           nag.py
           clipping.py
    datasets/               # Data handling
        __init__.py
        dataset.py
        file_dataset.py
```

```
        image_dataset.py
        transforms/
            min_max.py
nns/                          # Applications
   main.py
   conv2d.py
   playground.py
perceptron/                   # Simple examples
   main.py
docs/                         # Documentation
    ivus/
    medium/
    quarkml/
```

B Testing Guide

B.1 Running Tests

Execute the test suite with various options:

```
# Run all tests with verbose output
pytest -v

# Run specific test file
pytest core/test_gradient_check.py

# Run tests with coverage report
pytest --cov=core --cov-report=html

# Run tests matching a pattern
pytest -k "gradient"
```

B.2 Test Categories

The test suite includes:

- **Unit Tests**: Individual function and layer tests
- **Integration Tests**: Full model training tests
- **Gradient Tests**: Numerical verification of all components
- **Edge Cases**: Boundary conditions and error handling

This document and the associated codebase represent a comprehensive educational resource for understanding neural networks from first principles. All code is available under the MIT License.

Repository: github.com/Mondonno/nns